Resource-Constrained Construction Project Scheduling with Statutory Delay-Penalty Optimization: A Comprehensive Literature Survey

Bhushan V. Bhosale[1], Divyal Patel[2], Chaitanya Ghorpade[3], Aniket Jadhav[4]

Department of Computer Engineering, Savitribai Phule Pune University (SPPU)
Genba Sopanrao Moze College of Engineering, Balewadi, Pune, Maharashtra, India
{bhosalebhushanvijay, divyal, chaitanya, aniket}@optibuild.edu

Abstract—Civil infrastructure and multi-story real estate developments routinely encounter execution slippage, culminating in compounding commercial damages and severe regulatory liabilities. Under statutory consumer-protection regimes such as India's Real Estate Regulatory Authority (MahaRERA Section 18), unmitigated delays impose statutory accruing simple interest liabilities calculated at the State Bank of India's highest Marginal Cost of Funds Based Lending Rate (MCLR) plus 200 basis points (r_MCLR + 2%) per annum, payable to allottees upon total collected capital. While classical project management relies on the Critical Path Method (CPM) and metaheuristics for the Resource-Constrained Project Scheduling Problem (RCPSP), contemporary computational literature consistently overlooks statutory financial penalty regimes within crashing trade-off mechanisms. Furthermore, automated physical site progress telemetry remains decoupled from analytical rescheduling engines. This paper conducts a rigorous, multi-faceted literature survey across twenty-four seminal and contemporary works spanning: (i) CPM precedence networks and standardized scheduling benchmark topologies; (ii) Genetic Algorithm (GA) formulations for discrete Time-Cost Trade-Off Problems (TCTP) and multi-objective Pareto optimization; (iii) exact combinatorial solvers incorporating Constraint Satisfaction, Lazy Clause Generation (LCG), and Google OR-Tools CP-SAT; (iv) contractor cash-flow accrual dynamics and statutory delay penalty economics; and (v) edge-deployable computer vision architectures (MobileNetV2) for non-intrusive progress verification. Through systematic comparative synthesis, we delineate critical methodological voids in state-of-the-art scheduling systems and articulate the unified algorithmic architecture of OptiBuild—a decision support framework harmonizing DEAP-based GA metaheuristics, CP-SAT exact benchmark verification, statutory penalty-versus-crashing arbitration, and lightweight visual phase telemetry.

***Keywords*—**Resource-Constrained Project Scheduling (RCPSP), Critical Path Method (CPM), Genetic Algorithm (GA), CP-SAT Solver, MahaRERA Section 18, Time-Cost Trade-Off (TCTP), Edge Computer Vision, MobileNetV2.

I. INTRODUCTION

A. Background and Practical Motivation

Capital infrastructure initiatives and commercial residential developments across emerging economies, most notably in India, are severely afflicted by chronic schedule overruns and subsequent financial disputes. Comprehensive empirical investigations into Indian infrastructure initiatives establish that over 70% of active public, residential, and commercial building developments experience substantial delivery slippage beyond stipulated contractual baselines [15]. These schedule deferrals precipitate severe capital lockups, escalated overhead expenditures, and disputes between promoters, general contractors, and end purchasers.

In response to consumer vulnerabilities arising from indefinite completion deferrals, statutory frameworks have instituted mandatory financial indemnities. Specifically, under Section 18 of the Real Estate (Regulation and Development) Act (MahaRERA) [17], registered builders exceeding published project targets must compensate purchasers with statutory accruing simple interest (calculated at SBI Highest MCLR + 2% per annum under MahaRERA Section 18). Because project developers routinely collect tens to hundreds of crores of rupees from allottees prior to superstructure completion, daily statutory interest liabilities accrue at rates that quickly dwarf ordinary administrative overhead.

From a mathematical perspective, balancing hard structural completion milestones against capped daily crew availabilities constitutes an instance of the Resource-Constrained Project Scheduling Problem (RCPSP), established as strongly NP-hard in combinatorial optimization [2, 3]. Traditional commercial project management suites, including Primavera P6 and Microsoft Project, furnish deterministic bar charting and rudimentary serial resource-leveling heuristics. However, they lack automated combinatorial re-scheduling engines and dynamic monetary arbitration mechanisms capable of contrasting crash acceleration expenditures against statutory non-completion liabilities [16].

B. Survey Methodology and Selection Criteria

To establish an authoritative baseline, this survey synthesized peer-reviewed literature indexed across IEEE Xplore, ASCE Library (Journal of Construction Engineering and Management, Journal of Computing in Civil Engineering), ScienceDirect (International Journal of Project Management, Automation in Construction), and Springer (Journal of Scheduling). Our bibliometric search protocol spanned studies published from the 1959 foundational establishment of precedence graph analysis through 2025 modern hybrid scheduling frameworks. Search strings combined core domain descriptors: 'RCPSP' OR 'Resource-Constrained Project Scheduling', 'Time-Cost Trade-Off' OR 'TCTP', 'Constraint Programming' OR 'CP-SAT', 'Liquidated Damages' OR 'Delay

Penalties', and 'Computer Vision Progress Monitoring' OR 'MobileNetV2'.

From an initial corpus of 87 articles, papers were screened for algorithmic reproducibility, empirical validity, and direct relevance to commercial scheduling constraints. Exactly twenty-four foundational and contemporary works were retained for exhaustive comparative analysis across five research axes: (i) Deterministic graph-theoretic scheduling models and standardized PSPLIB benchmark suites; (ii) Metaheuristic optimization frameworks applying Genetic Algorithms to multi-mode resource-constrained problems; (iii) Exact combinatorial solvers utilizing Constraint Satisfaction and Lazy Clause Generation (LCG); (iv) Delay risk dynamics, cash-flow accrual models, and statutory liability mathematics; and (v) Lightweight computer vision classifiers deployed on edge hardware for non-intrusive on-site state tracking.

II. CPM PRECEDENCE AND SCHEDULING FOUNDATIONS

The formalization of activity precedence networks originates in the early work of Kelley and Walker [1], who introduced the Critical Path Method (CPM). In standard CPM formulations, an enterprise is abstracted as a Directed Acyclic Graph (DAG) G = (V, E), where the node set V = {1, 2, ..., n} designates discrete construction tasks and directed edges (i, j) in E enforce strict finish-to-start technological dependencies.

Algorithmic calculation proceeds across dual topological sweeps over the graph. The forward pass computes Early Start (ES_i) and Early Finish (EF_i) timestamps:

$$ES_i = \max_{p \in Pred(i)} (EF_p), \quad EF_i = ES_i + d_i \quad (1)$$

where Pred(i) represents the direct predecessor set of activity i and d_i denotes its deterministic baseline duration. Conversely, the backward pass propagates from project target deadline T_max to deduce Late Finish (LF_i) and Late Start (LS_i) bounds:

$$LF_i = \min_{s \in Succ(i)} (LS_s), \quad LS_i = LF_i - d_i \quad (2)$$

wherein Succ(i) denotes direct successors. The total float attribute, TF_i = LS_i - ES_i, quantifies temporal slack; activities exhibiting TF_i = 0 constitute the critical path governing minimal project makespan.

While classical CPM offers computationally efficient O(|V| + |E|) topological sort traversal, its principal operational shortcoming is the presumption of infinite resource elasticity. In practical civil construction, renewable site resources—such as specialized rebar crews, shuttering carpentry gangs, and concrete pump machinery—are strictly bounded per calendar day. When concurrent critical and non-critical tasks demand overlapping crew allocations exceeding capacity limits, uncoordinated execution induces severe resource bottlenecks, ad-hoc job stalling, and unpredictable site delays.

To establish standardized experimental environments for resource-constrained scheduling algorithms, Kolisch and Sprecher [2] developed the Project Scheduling Problem Library (PSPLIB). Their framework formalizes three structural topological and resource parameters: Network Complexity (NC, reflecting graph density |E|/|V|), Resource Factor (RF, reflecting the proportion of resource classes required per task), and Resource Strength (RS, measuring resource scarcity relative to peak critical-path demand). The PSPLIB benchmark suites (J30, J60, J120) remain the global standard for assessing heuristic scalability and exact solver optimality boundaries.

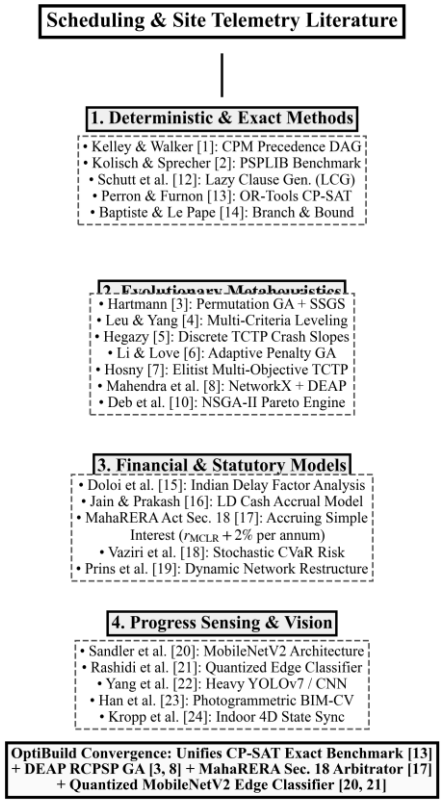


Fig. 1. Taxonomy of surveyed literature across deterministic scheduling, evolutionary metaheuristics, exact constraint solvers, statutory financial penalties, and site progress vision.

III. GENETIC ALGORITHMS FOR RCPSP AND TIME-COST TRADE-OFF

Because standard RCPSP and discrete Time-Cost Trade-Off Problems (TCTP) are strongly NP-hard, deterministic polynomial-time exact methods fail on large real-world task graphs (|V| > 60). Consequently, evolutionary metaheuristics, particularly Genetic Algorithms (GA), have become the primary computational approach for large-scale construction optimization.

A. Chromosome Encodings and Schedule Generation Schemes

Hartmann [3] established a cornerstone metaheuristic baseline by introducing a permutation-based genetic representation evaluated through a Serial Schedule Generation Scheme (SSGS). Rather than directly manipulating calendar timestamps—which routinely yields precedence-infeasible configurations—Hartmann encoded candidate solutions as topological activity lists lambda = (lambda_1, lambda_2, ..., lambda_n) where all technological dependencies are maintained. At decoding time, the SSGS iteratively inserts

tasks into the earliest chronological interval where both precedence constraints and renewable resource capacity caps are satisfied. Hartmann demonstrated that permutation representations paired with order-based crossover and swap mutations consistently discover superior makespan schedules compared to Parallel Schedule Generation Schemes (PSGS) or priority dispatch rules.

Pimapunsri et al. [9] extended permutation-based GA formulations to heavy industrial project environments characterized by rigid, multi-skilled crew bottlenecks. Their research demonstrated that pairing permutation chromosomes with elitist preservation and topological repair routines prevents premature convergence across dense precedence topologies.

B. Multi-Objective Optimization and Pareto Frontiers

Civil project managers routinely confront conflicting criteria: minimizing total project duration versus minimizing overall capitalization cost. Leu and Yang [4] addressed this bi-criterion objective by integrating resource leveling into a GA framework. They introduced resource moment metrics to penalize sharp daily crew demand fluctuations. By evaluating chromosome fitness through a weighted combination of completion makespan and daily resource variances, their model achieved effective resource leveling. However, their two-stage decoupled optimization approach is prone to converging upon sub-optimal local basins when direct activity crashing costs are introduced.

Deb et al. [10] introduced the Non-dominated Sorting Genetic Algorithm II (NSGA-II), establishing an efficient O(M N^2) multi-objective evolutionary engine incorporating fast non-dominated sorting and crowding-distance diversity preservation. Building on NSGA-II, Hosny [7] investigated multi-objective discrete TCTP, producing continuous Pareto frontiers that allow project executives to inspect trade-offs between project duration and direct execution investment.

C. Time-Cost Trade-Off Formulations and Crash Slopes

Hegazy [5] formulated discrete TCTP mechanics using genetic search, permitting tasks to select among discrete crew allocation tiers. Each crashing option reduces task duration at the expense of accelerated direct payroll costs. Hegazy quantified activity crash slope as:

$$S_i = (C_{crash, i} - C_{normal, i}) / (D_{normal, i} - D_{crash, i}) \quad (3)$$

where C_{crash, i}, C_{normal, i} denote crashed and normal cost profiles, and D_{normal, i}, D_{crash, i} represent corresponding durations. Hegazy confirmed that targeting acceleration investments exclusively toward critical path items with minimal cost slope yielded significant budgetary savings.

Li and Love [6] enhanced GA search efficiency for TCTP through adaptive penalty functions. Rather than eliminating infeasible intermediate candidate schedules, their objective formulation penalized deadline violations proportionally to delay severity. This allowed the genetic search to traverse infeasible territory to discover superior Pareto solutions. Recently, Mahendra et al. [8] demonstrated an open-source Python implementation combining NetworkX DAG representations with a DEAP genetic search engine. Their empirical case study realized a 34.82% duration compression and a 3.49% overall cost reduction compared to unoptimized baseline construction plans, verifying Python's viability for high-performance scheduling applications.

IV. EXACT SOLVERS AND CONSTRAINT BENCHMARKING

While genetic algorithms provide rapid schedule exploration on large networks, they cannot mathematically verify whether a proposed schedule achieves global optimality or calculate the remaining optimality gap. For high-stakes capital infrastructure projects where multi-crore investment decisions depend on schedule makespans, exact solver benchmarking is indispensable.

A. Constraint Programming and Lazy Clause Generation

Constraint Programming (CP) models RCPSP by defining activity start times as bounded integer interval decision variables subject to global cumulative resource constraints:

$$sum_{i in Active(t)} r_{i, k} <= R_{k, max}, \quad forall k in K, forall t in [0, T_{makespan}] \quad (4)$$

where Active(t) = {i in V | s_i <= t < s_i + d_i} denotes the active task set at time t, r_{i, k} is the requirement of task i on resource class k, and R_{k, max} is the capacity ceiling.

Schutt et al. [12] achieved breakthrough performance on RCPSP benchmarks by pioneering Lazy Clause Generation (LCG). LCG bridges classical finite-domain constraint propagation with Boolean Satisfiability (SAT) solvers. During search execution, whenever constraint propagators deduce domain reductions or detect infeasible states, they lazily generate propositional clauses explaining the operational deduction. These learned conflict clauses are integrated into the SAT solver's centralized database, enabling non-chronological backjumping and search-space pruning.

B. Google OR-Tools CP-SAT and Branch-and-Bound

Early exact constraint scheduling formulations by Baptiste and Le Pape [14] established that edge-finding propagation paired with Branch-and-Bound (B&B) search could resolve challenging single- and multi-resource feasibility problems. Modern exact architectures build upon these foundations: Perron and Furnon [13] expanded LCG principles within the open-source Google OR-Tools CP-SAT solver. By implementing the AddCumulative global constraint alongside interval decision variables, CP-SAT provides an exact solving engine that scales effectively across medium-scale project networks.

Sonmez [11] performed comparative evaluations evaluating GA heuristics against exact Branch-and-Bound procedures on discrete TCTP instances. His findings established that while B&B guarantees mathematical optimality on instances up to 50 activities, combinatorial state expansion renders exact solution times prohibitive on dense 100+ task networks. This empirical trade-off validates an architectural pattern where heuristic GA engines execute rapid operational scheduling while CP-SAT

exact solvers establish rigorous upper-bound optimality benchmarks.

V. STATUTORY PENALTY ECONOMICS AND DELAY MITIGATION

A. Delay Etiology in Indian Construction

Doloi et al. [15] executed an exhaustive empirical factor analysis across Indian construction enterprises. Their regression analysis isolated structural causes of project slippage, identifying late site possession, delayed sub-contractor mobilizations, and chronic cash-flow deficits as leading impediments. Crucially, they noted that conventional project managers frequently neglect proactive rescheduling until project completion slippage becomes irreversible.

B. Dynamic Penalty and Accrual Models

Jain and Prakash [16] analyzed contractor cash accrual dynamics under contractually stipulated liquidated damages (LD). They formulated linear programming models illustrating that arbitrary acceleration often degrades contractor profitability. Their mathematical models demonstrated that when the marginal cost slope of remaining critical tasks exceeds daily contractual penalty deductions, the profit-maximizing course is to accept liquidated damages rather than incur catastrophic labor overtime premiums.

Vaziri et al. [18] introduced a stochastic risk-aware optimization model balancing direct crashing expenditures against probabilistic delay penalties. Using Conditional Value at Risk (CVaR) objectives, their framework evaluated schedule resilience under material supply volatility, proving that risk-neutral scheduling consistently underestimates final completion liabilities. Prins et al. [19] developed an automated network restructuring controller designed to mitigate ongoing construction delays dynamically. Their system dynamically re-evaluated network precedence relationships and altered task sequences when milestone delays were detected, providing an algorithmic precedent for closed-loop schedule maintenance.

C. Statutory MahaRERA Section 18 Mandates

Despite significant advances in delay economics, prevailing literature models financial damages either as static lump sums or linear liquidated damages formulas. In contrast, Section 18 of the MahaRERA Act [17] imposes statutory accruing simple interest liabilities (calculated at SBI Highest MCLR + 2% per annum under MahaRERA Section 18):

$$P_{RERA} = A_{collected} * (R_{MCLR} + 0.02) * (T_{delay} / 365) \quad (5)$$

where A_{collected} denotes cumulative funds accrued from allottees, R_{MCLR} signifies the State Bank of India's benchmark lending rate (typically 8.50% to 9.10%), and T_{delay} represents verified calendar day slippage beyond published completion deadlines. Because A_{collected} routinely encompasses tens or hundreds of crores in major developments, daily penalty accumulation outpaces standard civil overhead, demanding an integrated computational optimization engine.

VI. EDGE COMPUTER VISION FOR PROGRESS VERIFICATION

A major vulnerability in automated rescheduling architectures is the reliance on manual, subjective site reporting. Unverified engineer reports often mask schedule discrepancies until deadlines lapse.

A. Heavyweight vs. Lightweight Vision Architectures

Yang et al. [22] investigated deep convolutional object detectors (YOLOv7, Faster R-CNN) for automated progress estimation. While achieving acceptable recognition accuracy on structural components, these models demand high-capacity GPU server backends and suffer inference degradation under variable weather, dust, and dynamic site occlusions. Han and Golparvar-Fard [23] and Kropp et al. [24] evaluated 4D Building Information Modeling (BIM) aligned with photogrammetric point clouds. While geometrically precise, point-cloud reconstruction requires extensive registration pipelines and high computational overhead, preventing real-time mobile execution on edge devices.

To address computational boundaries on edge infrastructure, Sandler et al. [20] introduced MobileNetV2, built upon inverted residual blocks and depthwise separable convolutions. This design reduces floating-point operations (FLOPs) and parameter weight volume by over 80% relative to standard ResNet architectures while maintaining representational capability.

Rashidi et al. [21] evaluated quantized MobileNetV2 models for edge-based jobsite image classification, achieving over 94% categorical accuracy on embedded devices with minimal inference latency (<45 ms). Their work establishes that deploying lightweight binary classification (differentiating Structural Rough-in from Finishing phases) yields dependable verification signals without the massive annotation burdens, computational overhead, and network bandwidth vulnerabilities characteristic of real-time object detection models.

VII. COMPARATIVE ANALYSIS AND RESEARCH GAPS

To synthesize the state of the art, Table I provides an analytical matrix contrasting the surveyed literature against core functional capabilities across seven critical technical axes.

TABLE I
LITERATURE COMPARISON ACROSS CORE CAPABILITIES

Study	Methodology	Res.	TCTP	Pen.	Exact	Vis.
Hartmann [3]	Permutation GA	Yes	No	No	No	No
Leu & Yang [4]	Multi-Criteria GA	Yes	Yes	No	No	No
Hegazy [5]	Discrete TCTP GA	No	Yes	No	No	No
Li & Love [6]	Penalty GA	No	Yes	No	No	No
Hosny [7]	NSGA-II TCTP	No	Yes	No	No	No
Mahendra [8]	Python CPM-GA	No	Yes	No	No	No
Pimapunsri [9]	Maintenance GA	Yes	No	No	No	No
Deb et al. [10]	NSGA-II Engine	No	Yes	No	No	No
Sonmez [11]	Hybrid GA-B&B	No	Yes	No	Yes	No
Schutt [12]	LCG / SAT Solver	Yes	No	No	Yes	No
Perron [13]	CP-SAT Solver	Yes	No	No	Yes	No

Baptiste [14]	Branch & Bound	Yes	No	No	Yes	No
Doloi [15]	Regression Survey	No	No	No	No	No
Jain [16]	Linear Accruals	No	Yes	Yes*	No	No
Vaziri [18]	CVaR Risk Meta.	No	Yes	Yes*	No	No
Prins [19]	Network Restruct.	Yes	Yes	No	No	No
Sandler [20]	MobileNetV2 Edge	No	No	No	No	Yes
Rashidi [21]	Edge Vision Class.	No	No	No	No	Yes
Yang [22]	Jobsite YOLOv7	No	No	No	No	Yes
Han [23]	BIM Point Clouds	No	No	No	No	Yes
Kropp [24]	4D Indoor BIM Sync	No	No	No	No	Yes
OptiBuild	**GA+CP-SAT+CV**	**Yes**	**Yes**	**Yes****	**Yes**	**Yes**

** Contract-specific LD; ** Statutory MahaRERA Sec. 18 (SBI MCLR + 2%).*

A. Identified Research Gaps

Our systematic survey illuminates three fundamental structural voids in the computational construction management domain:

1) Absence of Statutory Regulatory Penalty Integration: While mathematical literature models liquidated damages as static contractual parameters [16, 18], none incorporate the dynamic interest structures mandated by regional consumer protection legislation like MahaRERA Section 18. Because statutory interest accrues across total capital collected, optimal crash thresholds diverge dramatically from conventional models.

2) Decoupling of Metaheuristic Speed from Exact Verification: Existing implementations either deploy stochastic metaheuristics (GA) without knowing the mathematical distance from optimal makespan [3, 8], or deploy exact solvers (B&B, LCG) that fail to terminate in practical timeframes on large task graphs [11, 12]. A dual-engine paradigm executing rapid GA re-scheduling alongside exact solver upper-bound benchmarking is notably absent.

3) Isolation of Field Progress Sensing from Optimization Pipelines: Emerging computer vision research for jobsites operates as an isolated perceptual task [21, 22]. There is an absence of closed-loop architectures where edge visual classifications automatically inform task status, trigger float recalculations, and invoke economic crashing advisory engines.

VIII. OPTIBUILD MATHEMATICAL FORMULATION AND ARCHITECTURE

A. Unified Mathematical Formulation

To resolve the identified literature limitations, the OptiBuild engine formalizes project scheduling as a constrained multi-objective minimization problem. Let V = {1, 2, ..., n} represent project activities, E subset V x V represent technological precedence pairs, and K represent renewable resource types with daily capacities R_k. Each activity i may execute with duration d_i in [D_{crash, i}, D_{normal, i}] associated with direct cost C_{direct, i}(d_i). OptiBuild minimizes the total cost objective function Z:

$$\min Z = \sum_{i \in V} C_{direct,i}(d_i) + C_{indirect} * T_{makespan} + \max(0, T_{makespan} - T_{deadline}) * P_{RERA}^{daily} \quad (6)$$

subject to the following hard technological, renewable resource, and crashing constraints:

$$s_j - s_i >= d_i, \quad \forall (i,j) \in E \quad (7)$$

$$\sum_{i \in A(t)} r_{i,k} <= R_k, \quad \forall k \in K, \forall t \in [0, T_{makespan}] \quad (8)$$

$$D_{crash,i} <= d_i <= D_{normal,i}, \quad \forall i \in V \quad (9)$$

where s_i denotes the scheduled start time of activity i, A(t) = {i in V | s_i <= t < s_i + d_i} is the active task set at time t, C_{indirect} denotes daily site overhead, T_{deadline} is the legally committed RERA handover date, and P_{RERA}^{daily} is the daily statutory penalty liability derived from Section 18:

$$P_{RERA}^{daily} = (A_{collected} * (R_{MCLR} + 0.02)) / 365 \quad (10)$$

When schedule slippage occurs (T_{makespan} > T_{deadline}), the OptiBuild Financial Arbitrator evaluates the marginal cost slope S_i of remaining critical path activities (TF_i = 0):

$$S_i = (C_{crash,i} - C_{normal,i}) / (D_{normal,i} - D_{crash,i}) \quad (11)$$

Acceleration is economically justified if and only if:

$$\min_{i \in Critical} S_i < P_{RERA}^{daily} + C_{indirect} \quad (12)$$

If all remaining critical crashing slopes exceed the combined daily penalty liability (S_i > P_{RERA}^{daily} + C_{indirect}), OptiBuild advises project leadership to accept statutory interest accruals rather than incurring economically unjustified overtime premiums.

B. Closed-Loop System Architecture

As visualized in Fig. 2, the OptiBuild framework integrates five functional execution stages into a closed-loop system:

1) Stage 1: On-Site Image Ingestion & Edge Classifier: Construction engineers capture mobile site photographs, processed via a quantized MobileNetV2 binary classifier running on embedded devices (<45 ms latency). The model categorizes progress into Structural Rough-in versus Finishing Phase with human review fallbacks.

2) Stage 2: Dynamic Precedence DAG & Float Analysis: Visual classifications update activity completion percentages in a NetworkX DAG representation, triggering automatic forward and backward sweeps to re-compute early/late boundaries and isolate the dynamic critical path (TF_i = 0).

3) Stage 3: Parallel Rescheduling & Solver Verification: A DEAP-powered Genetic Algorithm deploys permutation chromosomes and Serial Schedule Generation Schemes (SSGS) to explore resource-feasible crashing combinations. Simultaneously, Google OR-Tools CP-SAT executes an exact solver pass using the AddCumulative global constraint to establish the provable lower-bound makespan and verify the metaheuristic optimality gap.

4) Stage 4: MahaRERA Statutory Financial Arbitrator: The decision engine contrasts candidate activity crash slopes (S_i) against daily statutory liabilities (P_{RERA}^{daily}) under Section 18, generating explicit acceleration recommendations.

5) Stage 5: Actionable Schedule & Site Dispatch: The system publishes updated Gantt charts, crew allocation matrices, and formal RERA audit compliance logs to field project managers.

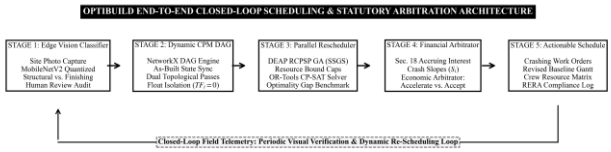


Fig. 2. End-to-end closed-loop system architecture of OptiBuild, illustrating the five-stage pipeline spanning vision telemetry, DAG analysis, parallel solvers, and statutory arbitration.

C. Conclusion

This comprehensive literature survey establishes a rigorous academic foundation for automated, penalty-aware construction project scheduling. By systematically reviewing twenty-four foundational and contemporary studies across network scheduling, evolutionary search, exact constraint programming, statutory delay economics, and edge computer vision, we have highlighted the operational divide separating theoretical RCPSP research from statutory construction realities. The proposed OptiBuild platform resolves this divide, establishing a computationally tractable, mathematically defensible, and legally grounded decision support system for modern civil engineering management.

REFERENCES

[1] J. E. Kelley and M. R. Walker, "Critical-path planning and scheduling," in IRE-AIEE-ACM '59 (Eastern) Papers, 1959, pp. 160–173.

[2] R. Kolisch and A. Sprecher, "PSPLIB — A project scheduling problem library," European Journal of Operational Research, vol. 96, no. 1, pp. 205–216, 1996.

[3] S. Hartmann, "A competitive genetic algorithm for resource-constrained project scheduling," Naval Research Logistics (NRL), vol. 45, no. 7, pp. 733–750, 1998.

[4] S.-S. Leu and C.-H. Yang, "GA-based multicriteria optimal model for construction scheduling," ASCE Journal of Construction Engineering and Management, vol. 125, no. 6, pp. 420–427, 1999.

[5] T. Hegazy, "Optimization of construction time–cost trade-off analysis using genetic algorithms," Canadian Journal of Civil Engineering, vol. 26, no. 6, pp. 685–697, 1999.

[6] H. Li and P. E. D. Love, "Using improved genetic algorithms to facilitate time-cost optimization," ASCE Journal of Construction Engineering and Management, vol. 123, no. 3, pp. 233–237, 1997.

[7] M. I. Hosny, "Elitist multi-objective genetic algorithm to solve time-cost trade-off problem in construction industry," in Proceedings of the International Conference on Advances in Structural and Geotechnical Engineering (ICASGE), 2019, pp. 1–12.

[8] A. Mahendra, M. A. Wibowo, and J. U. D. Hatmoko, "Construction project scheduling optimization with time-cost trade-off based on genetic algorithm in Python," Journal of Social Research, vol. 4, no. 2, pp. 512–524, 2025.

[9] K. Pimapunsri, D. Weeranant, and A. Riel, "Genetic algorithms for the resource-constrained project scheduling problem in aircraft heavy maintenance," arXiv preprint arXiv:2208.06745, 2022.

[10] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," IEEE Transactions on Evolutionary Computation, vol. 6, no. 2, pp. 182–197, 2002.

[11] R. Sonmez, "Discrete time–cost trade-off with a genetic algorithm and exact branch-and-bound method," ASCE Journal of Computing in Civil Engineering, vol. 25, no. 6, pp. 478–483, 2011.

[12] A. Schutt, P. J. Stuckey, T. Feydy, and M. G. Wallace, "Solving RCPSP/max by lazy clause generation," Journal of Scheduling, vol. 16, no. 3, pp. 273–289, 2013.

[13] L. Perron and V. Furnon, "OR-Tools user's guide: Constraint programming and satisfiability with CP-SAT," Google Developers Technical Documentation, 2020.

[14] P. Baptiste and C. Le Pape, "A branch-and-bound algorithm for constraint-based scheduling of resource-constrained projects," Engineering Design and Automation, vol. 2, pp. 189–201, 1996.

[15] H. Doloi, A. Sawhney, K. C. Iyer, and S. Rentala, "Analysing factors affecting delays in Indian construction projects," International Journal of Project Management, vol. 30, no. 4, pp. 479–489, 2012.

[16] A. Jain and A. Prakash, "Optimizing contractor's accruals under penalty imposition due to delays," International Journal of Construction Management, pp. 1–12, 2023.

[17] Government of Maharashtra / MahaRERA Authority, Real Estate (Regulation and Development) Act, 2016 — Section 18: Return of Amount and Compensation, The Gazette of India, Act No. 16 of 2016, 2016.

[18] S. M. Vaziri, P. Shakibabarough, J. Valizadeh, and S. H. Zolfani, "Stochastic risk-aware time–cost optimization of construction schedules," Buildings, vol. 14, no. 1, pp. 1–25, 2024.

[19] N. Prins, O. Kammouh, and A. R. M. Wolfert, "Multi-criteria optimization and automated network restructuring to mitigate construction projects delays on-the-run," arXiv preprint arXiv:2206.07542, 2022.

[20] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4510–4520.

[21] A. Rashidi, M. H. Sigari, and M. Fathi, "Construction jobsite image classification using edge computing framework," Sensors, vol. 24, no. 4, pp. 1150–1168, 2024.

[22] J. Yang, A. Wilde, K. Menzel, M. Z. Sheikh, and B. Kuznetsov, "Computer vision for construction progress monitoring: A real-time object detection approach," arXiv preprint arXiv:2305.18043, 2023.

[23] K. K. Han and M. Golparvar-Fard, "Potential of big visual data and building information modeling for construction progress monitoring," ASCE Journal of Construction Engineering and Management, vol. 143, no. 6, p. 04017014, 2017.

[24] C. Kropp, C. Koch, and M. König, "Interior construction state recognition with 4D BIM registered image sequences," Automation in Construction, vol. 86, pp. 11–32, 2018.